

Генерация промежуточного кода

Виды промежуточного кода

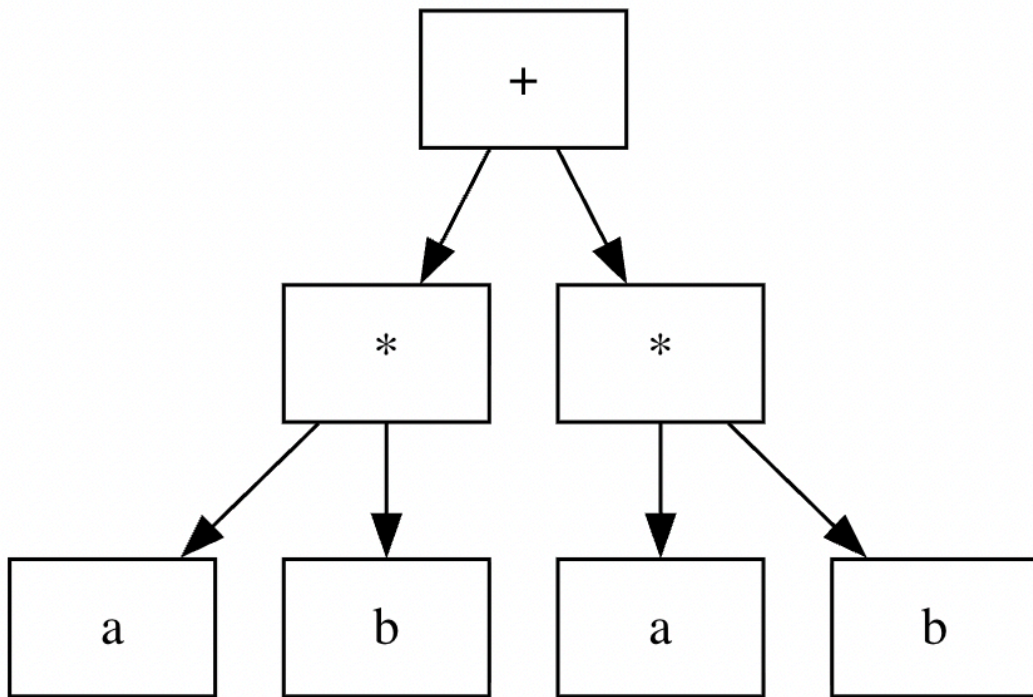
- Абстрактное синтаксическое дерево (AST)
- Ориентированный ациклический граф (DAG)
- Трехадресный код (TAC)

Абстрактное синтаксическое дерево

- Узлы - конструкции в исходной программе
- Дочерние узлы - значимые элементы конструкции

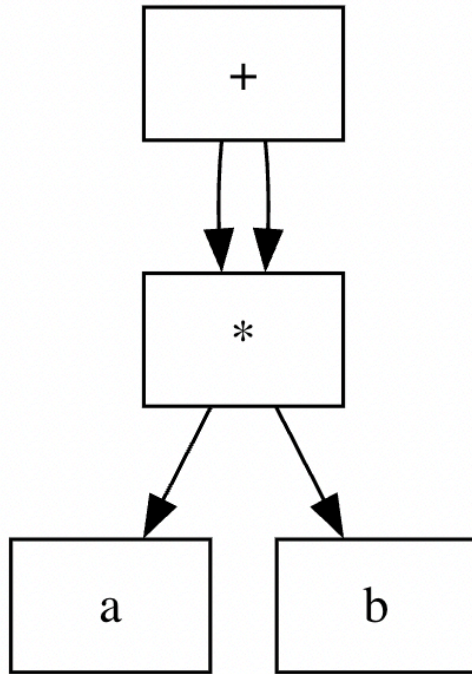
Пример

$a * b + a * b$



Ориентированный ациклический граф (DAG)

Объединяются общие подвыражения



Трехадресный код

```
x = y op z
```

- код команды
- адрес результата
- адрес первого и второго аргумента

Пример

$a * b + a * b$

```
t1 = a * b  
t2 = a * b  
t3 = t1 + t2
```

Набор команд ТАС (1)

- Арифметические и логические операции
 - `x = y op z` и `x = op y`
- Присваивания и перемещение данных
 - `x = y`
 - `x = *y` и `*x = y`
 - `x = y[i]` и `x[i] = y`
- Управление потоком
 - `goto L`
 - `if x relop y goto L`
 - `label L:`

Набор команд ТАС (2)

- Поддержка функций/процедур

- `param x`
- `call p, n`
- `x = call p, n`
- `return x`

- Специальные инструкции

- `x = &y`
- `x = y.f` и `x.f = y`

Static Single Assignment (SSA)

```
x := 10  
y := x + 5  
x := 20  
z := x + y
```

```
x1 := 10  
y1 := x1 + 5  
x2 := 20  
z1 := x2 + y1
```

Реализация трехадресных инструкций

- Четверки
- Тройки
- Косвенные тройки

Уровни промежуточного кода

Original

```
float a[10][20];  
a[i][j+2];
```

High IR

```
t1 = a[i, j+2]
```

Mid IR

```
t1 = j + 2  
t2 = i * 20  
t3 = t1 + t2  
t4 = 4 * t3  
t5 = addr a  
t6 = t5 + t4  
t7 = *t6
```

Low IR

```
r1 = [fp - 4]  
r2 = [r1 + 2]  
r3 = [fp - 8]  
r4 = r3 * 20  
r5 = r4 + r2  
r6 = 4 * r5  
r7 = fp - 216  
f1 = [r7 + r6]
```

Основные архитектуры виртуальных машин

- Стековая
- Регистровая

Стековая виртуальная машина

- `PUSH value`, `POP`
- `DUP`, `SWAP`
- `LOAD addr`, `STORE addr`
- `CALL addr`, `RET`

Пример (1)

```
PUSH 5          // n = 5
CALL fact       // вызов функции
PRINT           // вывод результата
HALT

fact:
  DUP           // дублировать n
  PUSH 1
  CMP           // n <= 1?
  JMPT base     // если да, базовый случай
```

Пример (2)

```
DUP          // иначе n * fact(n-1)
PUSH 1
SUB          // n-1
CALL fact
MUL          // n * fact(n-1)
RET
```

```
base:
POP          // убрать лишнее значение
PUSH 1       // вернуть 1
RET
```


Регистровая виртуальная машина

- Набор виртуальных регистров (обычно 16-256)

```
LOAD R0, address    // загрузить значение из памяти в регистр R0
ADD R1, R2, R3       // R1 = R2 + R3
STORE R1, address    // сохранить значение из R1 в память
```

Варианты в лабораторном практикуме (1)

- Байт-код JVM
 - JASM (<https://github.com/roscopeco/jasm>)
- Байт-код .NET
 - ILASM
- LLVM IR
 - llvm-as, llvmlite

Варианты в лабораторном практикуме (2)

- Байт-код CPython (.pyc)
 - python-xasm (<https://github.com/rocky/python-xasm>), python-reassembler
- WASM
 - WAT